COMP64401 Module 2: Propositional Logic

Structured Study Notes

Generated from the uploaded lecture transcripts and Week 2+3 slides

Contents

Topic and scope

Course: COMP64401 Logics for Knowledge Representation and Reasoning.

Lecture scope: Module 2 introduces propositional logic as the basic Boolean logic used for knowledge representation and reasoning, then develops satisfiability, validity, entailment, and a tableau-based SAT algorithm, before showing how these reasoning tasks support KR applications.

Source note: these notes were produced from the uploaded `Week2+3.pdf` slide deck and the two transcript files `2.1PropositionalLogic-English.txt` and `2.2PropositionalLogicCalculus-English.txt`. Sections 2.6-2.7 appear mainly in the slides, so those notes are based mainly on the slide content.

1. Organisation and study approach

The lecture begins with a reminder of how to approach the course unit:

- Watch the videos actively.
- Take notes while watching.
- Read up on the concepts afterwards.
- Bring questions to workshops.
- Use workshops to work through small tasks, ask questions, answer questions, and discuss.
- Work on coursework and use labs/GTAs for feedback.
- Use quizzes to test understanding and engage with feedback.

Exam flag / high-value study habit: the lecturer repeatedly stresses that technical terms, definitions, examples, lemmas, and theorems are the backbone of the course. Start a glossary, mind map, or large visual summary early and extend it throughout the semester. This is explicitly recommended for revision.

2. Propositional logic: syntax

2.1 What propositional logic is

Propositional logic is introduced as a basic logic, also known as **Boolean logic**. The lecturer assumes many students will have seen it before, but warns that different textbooks or previous courses may define the syntax slightly differently, so the exact definitions used in this course matter.

2.2 Propositional variables

Intuition: propositional variables are atomic statements whose truth values can be 0/false or 1/true.

Notation:

$$\mathcal{P} = \{p, q, r, ...\}$$

The set $\mathcal{P}$ contains propositional variables. The variables are often written as lowercase letters such as p, q, r, and may use subscripts such as p_i.

2.3 Propositional formulae

Formal definition: the set of propositional formulae over $\mathcal{P}$ is the **smallest** set such that:

1. Every propositional variable $p \in \mathcal{P}$ is a propositional formula.
2. If φ and ψ are propositional formulae, then the following are also propositional formulae:

$$\neg\varphi$$

$$\varphi \wedge \psi$$

$$\varphi \vee \psi$$

Parentheses are used to clarify precedence.

Intuition: the syntax tells us what expressions are legally allowed in the language. It does **not yet** say what those expressions mean.

2.4 Reading the symbols

The lecturer explicitly pauses to make sure the notation can be read aloud:

$$\neg\varphi$$

is read as "not φ".

$$\varphi \wedge \psi$$

is read as "φ and ψ".

$$\varphi \vee \psi$$

is read as "φ or ψ".

Greek letters used frequently:

$$\varphi \quad \text{phi}$$

$$\psi \quad \text{psi}$$

[UNCLEAR] The transcript often renders ψ as “CI”, “Cy”, or similar auto-transcription errors. These notes use ψ.

2.5 Inductive definitions

This syntax definition is an **inductive definition**: it builds formulae from smaller formulae.

The word **smallest** is important. It prevents arbitrary extra expressions from being counted as formulae. Only expressions generated by the rules are formulae.

Exam flag / common mistake: do not assume an expression is a propositional formula just because it looks logical. It must be generated by the syntax rules.

2.6 Syntax examples

The following is a propositional formula:

$$(p \wedge q) \vee \neg(q \vee r)$$

It is built using variables, conjunction, disjunction, negation, and parentheses.

The following is **not yet** a propositional formula under the basic syntax:

$$p \Rightarrow q$$

Reason: implication $\Rightarrow$ is not one of the primitive syntax operators yet.

The following is also **not** a propositional formula:

$$\forall x.\, p(x) \wedge q(x)$$

Reason: this uses quantification and predicates, which are not part of propositional logic as defined here.

3. Meta-observation: why definitions matter

A **definition** fixes the meaning of a technical term. The lecturer stresses that understanding technical terms is essential for understanding the rest of the course. Definitions allow us to classify examples correctly: formula vs non-formula, satisfiable vs unsatisfiable, valid vs not valid, and so on.

The recommended method for understanding a definition is:

1. Work through examples.
2. Re-read the definition carefully.
3. Work through more examples.
4. Ask questions if confusion remains.

The lecture connects this to Bloom's taxonomy: higher-level tasks like applying, analysing, evaluating, and creating depend on knowing and understanding the basic terms and concepts.

Exam flag: the lecturer explicitly says that understanding basic terminology is the basis for everything else. Definitions are not decorative; they are operational tools.

4. Propositional logic: semantics

4.1 Syntax vs semantics

Syntax tells us what can be legally written down.

Semantics gives meaning to formulae.

The semantics of propositional logic is given using **valuations**.

4.2 Valuation

Intuition: a valuation assigns a truth value to each propositional variable.

Formal definition: a valuation is a mapping

$$v : \mathcal{P} \to \{0, 1\}$$

where:

$$0 = \text{false}$$

$$1 = \text{true}$$

[UNCLEAR] The transcript says at one point "the set containing just the numbers one and two", but the slides and subsequent explanation clearly use $\{0, 1\}$. These notes use the slide definition.

4.3 Extending valuations from variables to formulae

A valuation is first defined on propositional variables. It is then extended inductively to all formulae:

$$v(\neg\varphi) := 1 - v(\varphi)$$

$$v(\varphi \wedge \psi) := v(\varphi) \cdot v(\psi)$$

$$v(\varphi \vee \psi) := \max(v(\varphi), v(\psi))$$

So:

- Negation flips 0 to 1 and 1 to 0.
- Conjunction is multiplication: it is 1 only if both sides are 1.
- Disjunction is maximum: it is 1 if at least one side is 1.

4.4 Making a formula true or false

A valuation v **makes** φ **true** if:

$$v(\varphi) = 1$$

A valuation v **makes** φ **false** if:

$$v(\varphi) = 0$$

The lecturer notes that, for full semantics, a valuation assigns a value to every propositional variable. Later algorithmic work may use partial valuations, but the semantic definition here uses full valuations.

5. Worked example: evaluating a formula under a valuation

Formula:

$$(p \wedge q) \vee \neg(q \vee r)$$

Take the valuation:

$$v(p) = v(q) = v(r) = 1$$

Evaluate subformulae:

$$v(q \vee r) = \max(v(q), v(r)) = \max(1, 1) = 1$$

$$v(\neg(q \vee r)) = 1 - v(q \vee r) = 1 - 1 = 0$$

$$v(p \wedge q) = v(p) \cdot v(q) = 1 \cdot 1 = 1$$

Therefore:

$$v((p \wedge q) \vee \neg(q \vee r)) = \max(1, 0) = 1$$

So this valuation makes the whole formula true.

The lecturer also gives a quicker “substitute values into the formula” view:

$$(p \wedge q) \vee \neg(q \vee r)$$

becomes:

$$(1 \wedge 1) \vee \neg(1 \vee 1)$$

Then:

$$1 \vee \neg 1$$

$$1 \vee 0$$

$$1$$

The lecturer points out that, for this formula, knowing p and q are true is already enough to make the whole formula true; r no longer matters in that case.

6. Truth tables

6.1 Definition and purpose

A truth table summarises valuations of a formula.

Standard structure:

- One row per valuation.
- One column per subformula.

For a formula with n propositional variables, the truth table has:

$$2^n$$

rows.

6.2 Truth table for $(p \wedge q) \vee \neg(q \vee r)$

p	q	r	$p \wedge q$	$q \vee r$	$\neg(q \vee r)$	$(p \wedge q) \vee \neg(q \vee r)$
1	1	1	1	1	0	1
1	1	0	1	1	0	1
1	0	1	0	1	0	0
1	0	0	0	0	1	1
0	1	1	0	1	0	0
0	1	0	0	1	0	0
0	0	1	0	1	0	0
0	0	0	0	0	1	1

Conclusion:

- The formula is **satisfiable**, because some rows evaluate to 1.
- The formula is **not valid**, because some rows evaluate to 0.

The slide on page 17 highlights this directly: the final column contains both 1s and 0s, so the formula is satisfiable but not valid.

7. Reasoning tasks

7.1 Satisfiability

Intuition: a formula is satisfiable if it can be made true by at least one valuation.

Formal definition: a propositional formula φ is **satisfiable** iff there exists a valuation v such that:

$$v(\varphi) = 1$$

This is an existential condition.

7.2 Validity

Intuition: a formula is valid if it is true under every valuation.

Formal definition: a propositional formula φ is **valid** iff for all valuations v:

$$v(\varphi) = 1$$

Valid formulae are also called **tautologies**.

Exam flag / common mistake: satisfiable means "true for at least one valuation"; valid means "true for every valuation". The lecturer explicitly contrasts existential vs universal quantification here.

7.3 Equivalence

Intuition: two formulae are equivalent if they always have the same truth value, even if they are written differently.

Formal definition: given propositional formulae φ, ψ, they are **equivalent**, written:

$$\varphi \equiv \psi$$

iff for all valuations v:

$$v(\varphi) = v(\psi)$$

The lecturer stresses that equivalence is not the same as syntactic identity. For example, $A \wedge B$ and $B \wedge A$ are not literally the same string, but they are equivalent.

7.4 Entailment

Intuition: φ entails ψ if every valuation that makes φ true also makes ψ true.

Formal definition used in the lecture:

$$\varphi \to \psi$$

iff for all valuations v:

$$v(\varphi) \leq v(\psi)$$

Since truth values are 0 and 1, this excludes exactly the counterexample case:

$$v(\varphi) = 1, \quad v(\psi) = 0$$

[UNCLEAR] The transcript wording around the possible truth-value cases for entailment is garbled. The formal inequality $v(\varphi) \leq v(\psi)$ is the reliable definition from the slides.

8. Useful equivalences and shortcuts

8.1 De Morgan-style equivalences

The lecture gives these examples:

$$\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi)$$

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi)$$

$$\neg\neg\varphi \equiv \varphi$$

These are later used when transforming formulae into negation normal form.

[UNCLEAR] The transcript appears to say “not not phi is equivalent to zero” at one point, but the slide and surrounding explanation give the standard equivalence $\neg\neg\varphi \equiv \varphi$.

8.2 Syntactic sugar

The primitive syntax only includes:

$$\neg, \quad \wedge, \quad \vee$$

Other operators can be defined for convenience.

Implication:

$$\varphi \Rightarrow \psi := \neg\varphi \vee \psi$$

Biconditional:

$$\varphi \Leftrightarrow \psi := (\varphi \Rightarrow \psi) \wedge (\psi \Rightarrow \varphi)$$

The lecture also notes that we do not strictly need both $\wedge$ and $\vee$, since each can be defined using the other plus negation.

Exam flag / common mistake: distinguish the semantic entailment relation $\varphi \rightarrow \psi$ from the object-language implication formula $\varphi \Rightarrow \psi$. The lecture uses both.

9. Why satisfiability and validity matter in KR&R

Suppose φ is a formula describing a system.

9.1 System descriptions should be satisfiable but not valid

A good system description should be:

satisfiable

because otherwise it is self-contradictory.

It should also be:

not valid

because if it is valid, it constrains nothing. It would be true under every valuation and therefore would not describe a meaningful restricted system.

The bicycle example on the slide illustrates this: if the system description is unsatisfiable, nothing could be a bicycle; if it is valid, everything would be a bicycle.

9.2 Desirable properties

If ψ is a desirable property of the system, then the system description should entail it:

$$\varphi \to \psi$$

Equivalently, the implication formula should be valid:

$$\varphi \Rightarrow \psi$$

Example from the slide:

$$\varphi \to \text{SafeRide}$$

The idea is that every model/valuation satisfying the system description also satisfies the desirable property.

9.3 Undesirable properties

If ψ is an undesirable property, then the conjunction of the system description with that property should be unsatisfiable:

$$\varphi \wedge \psi$$

should be unsatisfiable.

Example from the slide:

$$\varphi \wedge \text{RiskOfDeath}$$

should be impossible.

10. Reasoners and relationships between reasoning tasks

10.1 Reasoner

A **reasoner** is a piece of software that solves a reasoning task.

Examples:

- SAT solver: input φ, output yes/no for satisfiability.
- Validity solver: input φ, output yes/no for validity.
- Entailment solver: input φ, ψ, output yes/no for whether $\varphi \to \psi$.

10.2 Do we need four reasoners?

No. The lecture says we only need to build one, because reasoning tasks can be reduced to each other.

10.3 Theorem: relationships between reasoning tasks

Let φ, ψ be propositional formulae. Then:

1. φ is valid iff $\neg\varphi$ is not satisfiable.

$$\varphi \text{ valid} \quad \iff \quad \neg\varphi \text{ not satisfiable}$$

2. φ is satisfiable iff $\neg\varphi$ is not valid.

$$\varphi \text{ satisfiable} \quad \iff \quad \neg\varphi \text{ not valid}$$

3. $\varphi \to \psi$ iff $\varphi \Rightarrow \psi$ is valid.

$$\varphi \to \psi \quad \iff \quad \varphi \Rightarrow \psi \text{ valid}$$

4. $\varphi \to \psi$ iff $\varphi \wedge \neg\psi$ is not satisfiable.

$$\varphi \to \psi \quad \iff \quad \varphi \wedge \neg\psi \text{ not satisfiable}$$

The slide on page 27 visualises how to build multiple reasoners from one, e.g. a validity checker from a SAT solver by negating the input and swapping yes/no answers.

10.4 Building reasoners from a SAT solver

Validity using satisfiability

To decide whether φ is valid:

1. Ask SAT solver whether $\neg\varphi$ is satisfiable.
2. If SAT says yes, then φ is not valid.
3. If SAT says no, then φ is valid.

Entailment using satisfiability

To decide whether:

$$\varphi \to \psi$$

check whether:

$$\varphi \wedge \neg\psi$$

is satisfiable.

- If $\varphi \wedge \neg\psi$ is satisfiable, then there is a counterexample valuation: φ true and ψ false. So entailment fails.
- If $\varphi \wedge \neg\psi$ is unsatisfiable, no counterexample exists. So entailment holds.

Exam flag: this reduction is one of the most important technical moves in the lecture. It is how one SAT solver can support validity and entailment checking.

11. Truth tables as a reasoning method - and why they do not scale

A truth table can decide satisfiability, validity, equivalence, and entailment by enumerating valuations.

For a formula with n variables:

$$\text{number of rows} = 2^n$$

Example:

$$n = 10$$

gives:

$$2^{10} = 1024$$

rows.

The lecturer notes that real system descriptions, such as chip designs, can have hundreds of thousands of variables, making truth tables infeasible.

Exam flag / common mistake: truth tables are conceptually useful, but they are not a practical reasoning method for large KR&R applications. The lecture moves to calculi because truth tables are exponentially large by construction.

12. Summary so far

By the end of the first propositional logic part, the lecture has covered:

- Syntax: what a formula is.
- Semantics: valuations and truth/falsity.
- Reasoning tasks: satisfiability, validity, equivalence, entailment.
- Why reasoning tasks matter for KR&R.
- Technical terms and definitions.
- Lemmas and theorems as claims about concepts.
- Examples as a way to understand concepts.

The lecturer recommends beginning a glossary or mind map at this point. The slide on page 31 shows a handwritten mind-map-style diagram linking KR&R, logics, syntax, semantics, valuations, reasoning tasks, and related terms.

13. A calculus for propositional logic

13.1 Goal

The second part of the module introduces a calculus/algorithm for deciding satisfiability of propositional logic formulae.

The goal is to move beyond truth tables to a more goal-directed algorithm that can serve as the basis for a reasoner or SAT solver.

13.2 Tableau algorithm / analytical tableau

The algorithm is called:

- **tableau algorithm**
- **analytical tableau**

Intuition: to test whether φ is satisfiable:

1. Break down φ and its subformulae.
2. Explore choices created by disjunctions.
3. Try to find a valuation that makes φ true.

The lecture describes this as a kind of reasoning by cases for disjunctions.

Although tableau methods are often presented using trees, this lecture first presents the algorithm using **sets of formulae**:

- $\mathcal{S}$ is a set of sets of formulae.
- Each $S \in \mathcal{S}$ represents one possible valuation/path that might make the input formula true.
- Rules manipulate these sets.

14. Negation Normal Form

14.1 Definition

A formula is in **Negation Normal Form** - NNF - iff negation occurs only in front of propositional variables.

Example in NNF:

$$\neg p \land q$$

Example not in NNF:

$$\neg(p \land q)$$

because the negation is in front of a complex formula, not directly in front of a variable.

14.2 Why use NNF?

The tableau algorithm is simplified by first transforming the input formula into NNF. The lecture assumes only the operators:

$$\land, \quad \lor, \quad \neg$$

and then pushes negations inward. Having both $\land$ and $\lor$ available makes this transformation convenient using De Morgan-style rules.

14.3 NNF rewrite rules

Push negation inward using:

$$\neg(\varphi_1 \land \varphi_2) \rightsquigarrow \neg\varphi_1 \lor \neg\varphi_2$$

$$\neg(\varphi_1 \lor \varphi_2) \rightsquigarrow \neg\varphi_1 \land \neg\varphi_2$$

$$\neg\neg\varphi \rightsquigarrow \varphi$$

Apply these rules exhaustively, until none apply.

14.4 Lemma: NNF transformation

The lecture states the lemma:

Applying the above rewriting steps exhaustively transforms each propositional formula into one that is both:

1. equivalent to the original formula; and
2. in NNF.

The transcript also notes that this transformation does not blow up the formula size; the resulting formula is linear in the size of the original formula.

15. Worked example: converting to NNF

Start with:

$$\neg(\neg(p \wedge \neg(q \wedge r)) \vee q)$$

Apply De Morgan to the outer negated disjunction:

$$\rightsquigarrow \neg\neg(p \wedge \neg(q \wedge r)) \wedge \neg q$$

Eliminate double negation:

$$\rightsquigarrow (p \wedge \neg(q \wedge r)) \wedge \neg q$$

Push negation into $(q \wedge r)$:

$$\rightsquigarrow (p \wedge (\neg q \vee \neg r)) \wedge \neg q$$

Final formula:

$$(p \wedge (\neg q \vee \neg r)) \wedge \neg q$$

This is in NNF and equivalent to the original formula.

16. Tableau algorithm for PL satisfiability

16.1 Input and output

Input:

$$\varphi$$

where φ is a propositional formula in NNF.

Output:

- "yes" if φ is satisfiable.
- "no" otherwise.

16.2 Initialisation

Initialise a set of sets of formulae:

$$\mathcal{S} := \{\{\varphi\}\}$$

The double braces matter:

- $\{\varphi\}$ is a set containing the formula.
- $\{\{\varphi\}\}$ is a set containing that set.

16.3 The $\wedge$-rule

If there is some $S \in \mathcal{S}$ such that:

$$\varphi_1 \wedge \varphi_2 \in S$$

but:

$$\{\varphi_1, \varphi_2\} \nsubseteq S$$

then add both conjuncts to S:

$$S := S \cup \{\varphi_1, \varphi_2\}$$

Intuition: to make a conjunction true, both conjuncts must be true.

16.4 The $\vee$-rule

If there is some $S \in \mathcal{S}$ such that:

$$\varphi_1 \vee \varphi_2 \in S$$

but:

$$\{\varphi_1, \varphi_2\} \cap S = \emptyset$$

then replace S with two alternatives:

$$S \cup \{\varphi_1\}$$

and:

$$S \cup \{\varphi_2\}$$

Intuition: to make a disjunction true, it is enough to make one disjunct true. The algorithm therefore branches into cases.

16.5 Clash and clash-free sets

A set S contains a **clash** iff for some propositional variable p:

$$\{p, \neg p\} \subseteq S$$

A set is **clash-free** iff it does not contain a clash.

A clash is an obvious inconsistency: a valuation cannot make both p and $\neg p$ true.

16.6 Stopping condition and output

Apply the $\wedge$- and $\vee$-rules until no more rules apply.

Then:

- Return "yes" if $\mathcal{S}$ contains at least one clash-free set.
- Return "no" otherwise.

Exam flag: the return condition is existential: one clash-free final set is enough for satisfiability.

17. Worked example: tableau algorithm

Input:

$$\varphi = (p \wedge (\neg q \vee \neg r)) \wedge \neg s$$

This is already in NNF.

[UNCLEAR] The slide/transcript around this example appears inconsistent: the initial input uses $\neg s$, but some displayed copied sets show $\neg q$ in the top formula. The valuation examples also mention s. These notes use the initial formula $(p \wedge (\neg q \vee \neg r)) \wedge \neg s$.

Step 1: initialise

$$\mathcal{S} = \{\{(p \wedge (\neg q \vee \neg r)) \wedge \neg s\}\}$$

Step 2: apply $\wedge$-rule to the top-level conjunction

Add:

$$p \wedge (\neg q \vee \neg r)$$

and:

$$\neg s$$

So:

$$\mathcal{S} = \{\{\varphi,\ p \wedge (\neg q \vee \neg r),\ \neg s\}\}$$

Step 3: apply $\wedge$-rule again

Break down:

$$p \wedge (\neg q \vee \neg r)$$

Add:

$$p$$

and:

$$\neg q \vee \neg r$$

So:

$$\mathcal{S} = \{\{\varphi,\ p \wedge (\neg q \vee \neg r),\ \neg s,\ p,\ \neg q \vee \neg r\}\}$$

Step 4: apply $\vee$-rule

For:

$$\neg q \vee \neg r$$

split into two sets.

First set:

$$S_1 = \{\varphi,\ p \wedge (\neg q \vee \neg r),\ \neg s,\ p,\ \neg q \vee \neg r,\ \neg q\}$$

Second set:

$$S_2 = \{\varphi,\ p \wedge (\neg q \vee \neg r),\ \neg s,\ p,\ \neg q \vee \neg r,\ \neg r\}$$

So:

$$\mathcal{S} = \{S_1, S_2\}$$

Step 5: check clashes

Both S_1 and S_2 are clash-free.

Therefore the algorithm returns:

yes

The input formula is satisfiable.

Valuations extracted from the branches

From S_1:

$$v(p) = 1, \quad v(q) = 0, \quad v(s) = 0$$

The value of r does not matter.

From S_2:

$$v(p) = 1, \quad v(r) = 0, \quad v(s) = 0$$

The value of q does not matter.

Each clash-free set gives a valuation that makes the input formula true.

18. Alternative tree view of tableau

The set-of-sets presentation is easy to define formally, but the lecturer says it contains a lot of redundancy and is slow/wasteful.

A tree view avoids duplicating shared parts. In the tree:

- $\wedge$-rules extend the current branch.
- $\vee$-rules split the branch.
- Each full branch corresponds to one set S in $\mathcal{S}$.

For the example:

$$(p \wedge (\neg q \vee \neg r)) \wedge \neg s$$

the shared part of the branch contains:

$$p \wedge (\neg q \vee \neg r), \quad \neg s, \quad p, \quad \neg q \vee \neg r$$

Then the $\vee$-rule branches into:

$$\neg q$$

or:

$$\neg r$$

The slide around pages 42-43 illustrates this tree-style view and explicitly notes that the set-of-sets version has redundancy.

19. Correctness of the tableau calculus

19.1 Lemma: termination, soundness, completeness

Let φ be a PL formula in NNF. Then the tableau algorithm:

1. stops after at most:

$$2^{\ell(\varphi)}$$

steps;

2. returning “yes” implies φ is satisfiable;
3. returns “yes” if φ is satisfiable.

The slide labels these properties:

- **Termination:** always stops.
- **Soundness:** returning “yes” implies satisfiable.
- **Completeness:** if satisfiable, returns “yes”.

The lecturer summarises this as: the algorithm “always stops” and “never lies”.

19.2 Proof sketch: termination

The proof analyses the two rules.

For the $\wedge$-rule:

- It only adds formulae to some $S \in \mathcal{S}$.
- For each S, this happens at most once per subformula of φ.
- Therefore each S has size bounded by the length of the input formula:

$$\#S \leq \ell(\varphi)$$

Here $\#$ denotes cardinality.

For the $\vee$-rule:

- It replaces one set by two sets.
- The number of possible branchings is bounded exponentially by the number of disjunctions in φ.
- Thus the total number of generated sets is finite and exponential.

Once no rules apply, the algorithm stops.

19.3 Proof sketch: soundness

Assume the algorithm returns “yes”.

Then $\mathcal{S}$ contains a final clash-free set S.

Build a valuation v from S:

$$v(p) = 1 \quad \text{if } p \in S$$

$$v(p) = 0 \quad \text{if } \neg p \in S$$

If neither p nor $\neg p$ is in S, choose either value.

This valuation is well-defined because S is clash-free. If S contained both p and $\neg p$, it would try to assign both 1 and 0 to p.

Then prove, by induction on formula structure, that for every $\psi \in S$:

$$v(\psi) = 1$$

The original input formula φ remains in the set because the algorithm is non-destructive: formulae are added, not removed from branches. Hence:

$$v(\varphi) = 1$$

So φ is satisfiable.

19.4 Proof sketch: completeness

Assume φ is satisfiable.

Then there exists a valuation v such that:

$$v(\varphi) = 1$$

Use v to "watch" a branch through the rule applications:

- Start by watching:

$$S = \{\varphi\}$$

- If a $\wedge$-rule is applied, continue watching the extended set, because if:

$$v(\varphi_1 \wedge \varphi_2) = 1$$

then:

$$v(\varphi_1) = 1 \quad \text{and} \quad v(\varphi_2) = 1$$

- If a $\vee$-rule is applied to:

$$\varphi_1 \vee \varphi_2$$

then at least one of φ_1, φ_2 is true under v. Watch the branch that adds a true disjunct:

$$S \cup \{\varphi_1\} \quad \text{if } v(\varphi_1) = 1$$

otherwise:

$$S \cup \{\varphi_2\}$$

The final watched set must be clash-free, because a valuation cannot make both p and $\neg p$ true. Therefore the algorithm finds a clash-free set and returns "yes".

[UNCLEAR] The slide says "By contraposition: assume φ is satisfiable," but the transcript explicitly corrects this as a slide fluke. The proof is direct, not by contraposition.

19.5 Corollary

The tableau algorithm is:

1. a decision procedure for PL satisfiability;
2. exponential in time and space.

A **decision procedure** is an algorithm that decides a yes/no problem: it terminates and gives correct answers.

20. Implementation and SAT solvers

The tableau algorithm as presented is conceptually useful but inefficient.

To implement it well, one needs clever design decisions:

- suitable syntax for formulae;
- efficient data structures;
- good order of rule applications;
- choice of which branch to pursue first;
- ways to learn from failed branches;
- optimisations to avoid wasteful duplication.

The lecturer connects this to practical SAT solvers such as MiniSAT and the SAT competition. Although propositional satisfiability is NP-complete/intractable in the worst case, SAT solvers can be very successful in practice because optimisations often avoid worst-case behaviour on formulae of interest.

Exam flag: hard in theory does not mean useless in practice. The lecturer explicitly highlights that SAT remains useful because clever implementations often avoid worst-case behaviour.

21. Using reasoning in KR applications

21.1 Two ways to use a reasoner

The lecture distinguishes two application styles.

On-stage usage

Users interact directly with the SAT solver.

This is rare and mainly occurs in settings such as teaching tools.

It requires:

- suitable syntax;
- user interface;
- interaction mechanisms.

Behind-the-scenes usage

Users do not see the SAT solver.

Instead:

1. the user's task is translated into a SAT problem;
2. the SAT solver solves it;
3. the answer is translated back into the original user task.

The slide gives program verification as an example. The diagram on page 52 illustrates a SAT solver either being directly on stage or hidden behind an application layer.

22. KR example: weather knowledge base

22.1 Propositional variables

The slide introduces propositional variables for weather and human behaviour:

Meaning	Variable
Blue Sky	BS
Sunny	Sy
Raining	Rg
Snowing	Sg
Street Wet	SW
Below Zero	BZ
Cold Weather	CW
Slippery	Sly
Sit Outside	SO
Take Umbrella	TU
Take Coat	TC
Wear Boots	WB
Rainbow	RB

These variables are used to build a knowledge base describing weather and some human behaviour.

22.2 Example formulae

The slide lists formulae such as:

$$BS \Rightarrow Sy$$

If there is blue sky, then it is sunny.

$$Rg \Rightarrow SW$$

If it is raining, then the street is wet.

$$BZ \Rightarrow CW$$

If it is below zero, then it is cold weather.

$$(Rg \wedge Sy) \Rightarrow RB$$

If it is raining and sunny, then there is a rainbow.

$$(Sg \vee (SW \wedge BZ)) \Rightarrow Sly$$

If it is snowing, or the street is wet and it is below zero, then it is slippery.

$$(Sly \vee CW) \Rightarrow (WB \wedge TC)$$

If it is slippery or cold, then wear boots and take a coat.

$$(Rg \vee \neg Sy) \Rightarrow TU$$

If it is raining or not sunny, then take an umbrella.

$$CW \Rightarrow \neg SO$$

If it is cold weather, then do not sit outside.

[UNCLEAR] The slide uses RG in one place and Rg elsewhere. These notes use Rg consistently for “Raining”.

22.3 Knowledge base as conjunction

The whole knowledge base is treated as a conjunction of formulae:

$$KB = (1) \wedge (2) \wedge \cdots \wedge (19)$$

The lecture then asks: when are we done, and are the formulae any good?

23. Reasoning checks for the weather KB

23.1 Sanity check 1: satisfiability

Question:

$$KB \text{ satisfiable?}$$

If yes:

consistent theory

If no:

broken theory

This checks that the knowledge base is not self-contradictory.

23.2 Sanity check 2: validity

Question:

$$KB \text{ valid?}$$

If yes:

vacuous theory

This is bad, because the KB constrains nothing.

If no:

constraining theory

This is good.

23.3 Entailment checks: are entailed formulae good?

Example:

$$KB \to (BZ \Rightarrow \neg SO)$$

If yes, the KB entails that if it is below zero, one should not sit outside.

If no, the slide suggests weakening or fixing some formulae.

23.4 Which entailments should be checked?

The lecture says checking all entailments is impossible, because even simple formulae have infinitely many entailments.

Example pattern:

$$(p \vee \neg p) \to (q_1 \vee \neg q_1)$$

$$\to (q_2 \vee \neg q_2)$$

$$\to \dots$$

and conjunctions of arbitrarily many tautologies.

So tools should focus on interesting entailments and avoid:

- tautologies;
- formulae already contained in the KB;
- equivalent entailments;
- showing too many results.

The slide labels this “Really tricky & interesting!”

23.5 Desired-property checks

Question:

$$KB \to (BZ \Rightarrow \neg SO)$$

interpreted as: does the KB entail a desired property?

If yes:

KB reflects desired properties

If no:

strengthen/fix some formulae

The slide notes that we need to collect desired property formulae.

23.6 Undesired-property checks

Question:

$$KB \wedge (Rg \wedge \neg TU)$$

is unsatisfiable?

This checks whether the KB rules out an undesired situation: it is raining, but the agent does not take an umbrella.

If yes:

KB is inconsistent with undesired properties

That is good: the bad situation cannot happen.

If no:

strengthen/fix some formulae

The slide notes that we need to collect undesired property formulae.

24. Tooling tasks for knowledge-base design

To use automated reasoning well for designing a knowledge base, the lecture says we need clever tool support and methodologies for:

24.1 Vocabulary choice and maintenance

Including:

- term definitions;
- alternative terms;
- terms in other languages;
- vocabulary maintenance.

24.2 Knowledge-base design and maintenance

Including:

- interaction with the reasoner;
- gathering desired and undesired properties;
- explaining reasoner results;
- fixing, strengthening, or weakening formulae in the KB.

24.3 Documentation and versioning

Including:

- documentation of design choices;
- versioning of knowledge bases.

The slide asks: “What do we want to use KB for?” This matters because the intended use determines which checks and tool support are needed.

25. Limitations of propositional logic in KR applications

The final slide asks what happens if we want more detailed weather knowledge.

25.1 Locations

Propositional logic struggles to express general location-dependent patterns compactly.

For example, we might need separate variables:

$$BS_{Man}$$

$$BS_{Bir}$$

for blue sky in Manchester vs Birmingham.

Then rules also proliferate:

$$BS_{Man} \Rightarrow Sy_{Man}$$

$$BS_{Bir} \Rightarrow Sy_{Bir}$$

and so on.

The slide hints that something like quantification would be more natural:

$$\forall x.\ BS(x) \Rightarrow BS(y)$$

[UNCLEAR] This displayed formula has y free/unbound, so it is likely either a slide typo or an intentionally rough placeholder. The point is that propositional logic lacks variables and quantification over locations.

25.2 Time

The slide mentions today, tomorrow, yesterday, and statements such as:

> eventually it will stop raining

It also gives an example involving:

$$\neg TU \Rightarrow soon(Rg)$$

This is outside plain propositional logic because it involves a temporal expression, $soon$.

25.3 Probabilities

The slide mentions probabilistic weather statements, such as:

$$p \geq 95$$

[UNCLEAR] The slide text is truncated around “it will rain with a …”; this likely refers to a probability threshold, but the exact intended wording is not fully visible in the parsed text.

25.4 Beliefs

The slide also mentions beliefs about weather, with an example like:

$$\neg TU \Rightarrow Rg$$

inside a belief statement:

> I believe that …

This points beyond propositional logic toward logics that can represent beliefs.

The slide says these richer topics come in the next sessions.

26. Connections made in the lecture

26.1 Connection to previous learning

The lecturer notes that students may already know propositional logic from undergraduate study or under the name Boolean logic. However, previous courses or textbooks may define the syntax differently, for example by including implication as a primitive operator. The course's own definitions are the reference point.

26.2 Connection to algorithms and SAT solvers

The tableau calculus is presented as a basis for a SAT reasoner. Practical SAT solvers use data structures and heuristics beyond the simple formal presentation. MiniSAT and SAT competitions are mentioned as examples of successful SAT-solver work.

26.3 Connection to knowledge representation

The weather example shows how formulae can represent domain knowledge and how reasoning tasks can test whether the KB is consistent, non-vacuous, and aligned with desired/undesired properties.

26.4 Connection to later topics

The limitations slide points forward to richer logics for:

- objects and quantification;
- time;
- probability;
- belief.

27. Consolidated exam flags / high-value points

No explicit "this will be on the exam" phrase appears in the provided transcript. The following are marked because the lecturer explicitly emphasises them, repeats them, or uses them as core technical machinery.

1. **Know the definitions exactly.** Technical terms are central: formula, valuation, satisfiable, valid, equivalent, entails, clash, NNF, soundness, completeness, termination.
2. **Distinguish syntax from semantics.**
   - Syntax: what can be written.
   - Semantics: what it means.
3. **Distinguish satisfiable from valid.**

- Satisfiable: $\exists v$ such that $v(\varphi) = 1$.
- Valid: $\forall v$, $v(\varphi) = 1$.

4. **Remember that valid formulae are tautologies.**
5. **Do not confuse equivalence with syntactic identity.**
   - $A \wedge B$ and $B \wedge A$ are different strings but equivalent.
6. **Distinguish entailment $\rightarrow$ from implication $\Rightarrow$.**
   - $\varphi \rightarrow \psi$: semantic relation.
   - $\varphi \Rightarrow \psi := \neg\varphi \vee \psi$: formula abbreviation.
7. **Know the reduction theorem.**
   - Validity, satisfiability, and entailment can be reduced to each other.
   - One reasoner can be used to build others.
8. **Truth tables scale exponentially.**
   - n variables gives 2^n rows.
   - Useful conceptually, not practical for large systems.
9. **Know NNF rewrite rules.**

$$\neg(\varphi_1 \wedge \varphi_2) \rightsquigarrow \neg\varphi_1 \vee \neg\varphi_2$$

$$\neg(\varphi_1 \vee \varphi_2) \rightsquigarrow \neg\varphi_1 \wedge \neg\varphi_2$$

$$\neg\neg\varphi \rightsquigarrow \varphi$$

10. **Know the tableau rules.**
    - $\wedge$-rule: add both conjuncts.
    - $\vee$-rule: branch into alternatives.
11. **Know clash-free criterion.**
    - A set with both p and $\neg p$ has a clash.
    - One final clash-free set is enough for satisfiability.
12. **Know correctness vocabulary.**
    - Termination: stops.
    - Soundness: yes-answer is trustworthy.
    - Completeness: all satisfiable inputs get yes.
    - Together they give a decision procedure.
13. **Know why KR system descriptions should be satisfiable but not valid.**
    - Unsatisfiable: inconsistent/broken.
    - Valid: vacuous/unconstraining.
14. **Know desired vs undesired property checks.**
    - Desired: check entailment.
    - Undesired: check unsatisfiability of KB plus bad property.
15. **Theory vs practice.**
    - PL satisfiability is NP-complete/intractable in the worst case.
    - SAT solvers can still be practically useful with good heuristics and data structures.

28. Unclear sections to revisit in the recording/slides

1. ψ **transcription:** the transcript repeatedly renders ψ as “CI”, “Cy”, or similar.
2. **Valuation codomain:** the transcript says “numbers one and two,” but the slides and formulas use $\{0, 1\}$.
3. **Double negation equivalence:** the transcript appears to say $\neg\neg\varphi$ is equivalent to “zero”; the slide and logic of the discussion give $\neg\neg\varphi \equiv \varphi$.

4. **Entailment wording:** the transcript garbles the truth-value cases; rely on $v(\varphi) \leq v(\psi)$.
5. **Tableau example formula:** the input appears as $(p \wedge (\neg q \vee \neg r)) \wedge \neg s$, but some copied sets show $\neg q$ in the top formula. The valuation examples support the $\neg s$ version.
6. **Completeness proof label:** the slide says "By contraposition" while assuming φ is satisfiable. The transcript explicitly corrects this as a slide mistake; the proof is direct.
7. **Weather rule notation:** RG and Rg both appear; likely the same "Raining" variable.
8. **Limitations slide formula:** $\forall x.BS(x) \Rightarrow BS(y)$ contains an unbound y, so revisit the recording if this was discussed.
9. **Probability example:** the slide text around "it will rain with a ... $p \geq 95$" is truncated/garbled.